

Pin-to-Pin System Architecture, Class Design & Master Technical Interview Guide

Tech Stack: Java, Jakarta EE (Servlets & JSP), JDBC, MySQL 8.0, Apache Tomcat 10+, HTML5, CSS3, JavaScript, Eclipse IDE

Reference Models: Myntra, Ajio, Nykaa (Fashion, Multi-Size Footwear & Beauty Cosmetics)

Author / Engineer: Indranil Soma | Portfolio & Campus Recruitment Specification

1. High-Level Enterprise Architecture & Request Lifecycle

TrendVibe is engineered around the industry-standard **Model-View-Controller (MVC)** and **Data Access Object (DAO)** architecture. This strictly decouples presentations, routing logic, business validation, and database queries for enterprise maintainability.

```
[Client Web / Mobile Browser]
    ↓ (HTTP Requests / Form Submission / JSON)
[Apache Tomcat 10+ Web Server] → [web.xml Deployment Descriptor]
    ↓
[Controller Layer (Jakarta Servlets)]
    • ProductController (/home, /products, /product-details)
    • CartController (/cart, /cart/add, /cart/update, /cart/remove)
    • CheckoutController (/checkout, /checkout/place-order, /my-orders)
    • LoginController / RegisterController (/login, /register)
    ↓
[DAO Layer (Data Access Objects & JDBC)]
    • DBConnection (Connection Pool & Driver Manager)
    • ProductDAO, CartDAO, OrderDAO, UserDAO, CategoryDAO
    ↓ (Atomic SQL Queries & PreparedStatements)
[MySQL Relational Database (ecommerce_db)]
    • 8 Normalized Tables with Foreign Keys, Cascades & Indexing
```

What is the 'Main File' in this application?

Unlike standalone C/Java programs that execute from public static void main(String[] args), an enterprise Jakarta EE web application contains **four primary architectural entry points**:

1. **Configuration Entry Point (web.xml)**: Located in WEB-INF/web.xml. This is the very first file Tomcat reads to register servlets, URL patterns, session timeouts, and the welcome list.
2. **Web Request Entry Point (index.jsp)**: The root welcome file that intercepts incoming traffic (/) and forwards to /home.
3. **Primary Controller Entry Point (ProductController.java)**: Dispatches home catalog feeds, category sorting, and product detail views.
4. **Primary Database Entry Point (DBConnection.java)**: Centralizes JDBC connection creation to MySQL.

2. Pin-to-Pin Class Creation & Object-Oriented Hierarchy

To satisfy the design blueprint, the application is divided into three Java packages under com.ecommerce:

Package / Layer	Class Name	Responsibility & Core Methods
com.ecommerce.model	User, Product, Category, ProductSize, Order, OrderItem, Cart, CartItem	JavaBeans (POJOs) implementing Serializable. Encapsulates private fields with getters/setters and calculated pricing methods.
com.ecommerce.dao	DBConnection	Thread-safe connection provider. Loads com.mysql.cj.jdbc.Driver, reads environment variables (Host, Port, User, Password) with failover.
com.ecommerce.dao	ProductDAO, CartDAO, OrderDAO, UserDAO, CategoryDAO	Executes prepared statements, converts ResultSet rows into domain model objects, manages batch insertions, and runs atomic SQL updates.
com.ecommerce.controller	ProductController, CartController, CheckoutController, etc.	Extends HttpServlet. Overrides doGet() and doPost(). Sanitizes input, verifies session state, and forwards to JSP views.

3. Multi-Size Inventory Management & Concurrency Control

A major challenge in fashion e-commerce (such as Myntra, Ajio, and Nykaa) is tracking stock across multiple variations (sizes like S, M, L, XL, or shoe sizes UK 7, UK 8). Storing stock on the product level fails because size 'M' may be sold out while size 'XL' remains available.

Relational Schema Implementation:

We maintain a dedicated product_sizes table with a composite key logic: product_id (FK), size_label, stock_quantity, and unique sku_code. Every cart item links specifically to a (product_id, size_label) pair.

How to Prevent Overselling During Flash Sales (Concurrency & Locking):

When 100 users try to buy the last 2 units of a Nike sneaker simultaneously, naive code causes race conditions (dirty reads and overselling). In our application, we handle this through **ACID-compliant Database Transactions**:

```
// Atomic Stock Decrement within a Transaction (OrderDAO.java)
connection.setAutoCommit(false); // Begin ACID Transaction

String stockSql = "UPDATE product_sizes "
    + "SET stock_quantity = stock_quantity - ? "
    + "WHERE product_id = ? AND size_label = ? "
    + "AND stock_quantity >= ?"; // Guard Condition!

try (PreparedStatement ps = connection.prepareStatement(stockSql)) {
    ps.setInt(1, item.getQuantity());
    ps.setInt(2, item.getProductId());
    ps.setString(3, item.getSizeLabel());
    ps.setInt(4, item.getQuantity()); // Must have at least this amount

    int rowsAffected = ps.executeUpdate();
    if (rowsAffected == 0) {
        // Stock was insufficient! Abort checkout.
        connection.rollback();
        throw new OutOfStockException("Size " + item.getSizeLabel() + " is sold out!");
    }
}
connection.commit(); // Successfully updated and locked
```

Pessimistic Locking vs. Optimistic Locking:

- **Pessimistic Locking (SELECT ... FOR UPDATE):** Acquires an exclusive row lock at the database level. No other transaction can read or write that row until the transaction commits. Ideal for high-contention flash sales.
- **Optimistic Locking (Version Column):** Each row has a version INT. Updates only succeed if WHERE version = current_version. If another buyer updated first, a StaleObjectStateException is thrown and the user retries.

4. Top 25 Technical Interview Questions & High-Scoring Answers

The following questions are specifically tailored to this project and represent what top recruiters and senior engineers ask in Java, E-Commerce, and System Design interviews:

Q1. Walk me through the architecture of your e-commerce project.

I developed TrendVibe, an enterprise fashion and cosmetics e-commerce platform inspired by Myntra and Nykaa. It follows the MVC and DAO architectural patterns. The View layer utilizes responsive HTML5, CSS3, and JSP with JSTL. The Controller layer uses Jakarta EE Servlets mapped via web.xml to handle requests and manage HTTP sessions. The Model layer consists of serializable JavaBeans, while the DAO layer executes parameterized JDBC queries against a normalized 8-table MySQL database. I deployed the web application on Apache Tomcat 10+.

Q2. Why did you choose the DAO pattern instead of writing SQL queries directly inside Servlets?

Writing SQL queries in Servlets violates the Single Responsibility Principle and couples web request handling with database access. The DAO pattern abstracts and encapsulates all database interactions. If we decide to migrate from MySQL to PostgreSQL, or adopt an ORM like Hibernate, we only modify the DAO implementation classes without changing a single line of servlet or controller code.

Q3. How do you prevent SQL Injection attacks in your application?

We strictly use PreparedStatement with parameterized placeholders (?) instead of string concatenation in raw SQL statements. PreparedStatements pre-compile the SQL statement on the database server. User input is treated strictly as literal data rather than executable SQL code, entirely neutralizing injection attempts.

Q4. How does the shopping cart work for guest users vs. authenticated users?

When an unauthenticated guest user adds an item to their bag, the cart is maintained in their HTTP session (session.getAttribute("cart")). Once the user logs in or registers, the session cart items are merged into the persistent MySQL cart and cart_items tables associated with their unique user_id. This ensures their selections are never lost across devices.

Q5. How do you manage database connections efficiently? What happens under high load?

Instead of opening and closing a physical connection on every HTTP request—which is expensive due to TCP handshake and authentication overhead—we utilize connection pooling via DBConnection. We reuse pre-established database connections from the pool. In high-traffic scenarios, Tomcat's JNDI DataSource maintains an active pool (e.g. 20-50 connections), drastically lowering response latency.

Q6. How does your system handle multi-size inventory (e.g. Shirts with S, M, L, XL)?

We designed a normalized one-to-many relationship: products (1) → product_sizes (N). Each size entry holds its own size_label, stock_quantity, and unique sku_code. This allows us to display real-time stock availability per size, alert users when a specific size has 'Only 2 units left', and disable sold-out sizes dynamically in the UI.

Q7. Explain how an order checkout transaction works and how you guarantee ACID properties.

Checkout involves multiple database operations: inserting into orders, inserting multiple rows into order_items, decrementing stock in product_sizes, and clearing cart_items. We invoke connection.setAutoCommit(false) to begin an atomic transaction. If any item is out of stock or an error occurs, we execute connection.rollback(), guaranteeing that stock is never deducted without an order being created. If all succeed, connection.commit() commits the changes.

Q8. How did you ensure the website is 100% mobile-responsive for smartphones?

I used a mobile-first CSS architecture. For viewports under 768px, the product grid automatically formats into a native-feeling 2-column layout (grid-template-columns: repeat(2, 1fr)) with 4:5 fashion aspect ratios. We also implemented a slide-out drawer menu, horizontally scrolling category chips, and a native app bottom navigation bar with iOS safe-area inset support.

Q9. What is the difference between forward() and sendRedirect()?

`RequestDispatcher.forward()` happens entirely on the server-side: Tomcat forwards the request and response objects to another resource (like a JSP view) without the client knowing, keeping the original URL in the address bar. `HttpServletResponse.sendRedirect()` sends an HTTP 302 redirect header to the client browser, forcing it to initiate a new GET request to the new URL. We use forward for rendering views and sendRedirect after successful POST actions (Post/Redirect/Get pattern to prevent duplicate form submissions).

Q10. What is the Servlet Lifecycle?

Tomcat manages the servlet lifecycle through three core methods: 1. `init(ServletConfig config)`: called once when the servlet is first loaded into memory. 2. `service(HttpServletRequest req, HttpServletResponse res)`: called on every client request, which delegates to `doGet()`, `doPost()`, etc., across worker threads. 3. `destroy()`: called once when the server shuts down or the application is undeployed to release resources.

Q11. Are Servlets thread-safe?

No. Tomcat creates a single instance of each servlet and executes concurrent requests on separate threads using the same instance. Therefore, instance variables inside a servlet are shared across all users and threads, leading to race conditions. To ensure thread safety, servlets must be stateless: all request-specific data must be stored in local method variables, request attributes, or session attributes.

Q12. What indexes did you create in MySQL and why?

We added B-Tree indexes on high-frequency query columns: `category_id` on products, `product_id` on product_sizes, `user_id` on orders, and `cart_id` on cart_items. Without indexing, MySQL performs full table scans ($O(N)$ complexity). With indexes, lookup time is reduced to $O(\log N)$, allowing instant catalog filtering and cart retrieval.

Q13. How would you scale this platform to handle 1 Million daily active users?

I would implement horizontal scaling and caching: 1. Place an Nginx reverse proxy / load balancer in front of multiple Tomcat nodes. 2. Use Redis as a distributed in-memory cache for product catalog queries and session storage. 3. Configure MySQL Master-Slave replication (writes to Master, reads distributed across Read Replicas). 4. Offload static media (images, CSS, JS) to a Content Delivery Network (CDN) like AWS CloudFront or Cloudflare.

Q14. How did you structure your Git repository and deployment workflow?

The project uses standard Git branching. The main branch contains the complete production-ready Jakarta EE and Maven source code for Eclipse IDE. We automated continuous deployment by publishing an interactive, live client-side application to the gh-pages branch on GitHub Pages, giving recruiters instant access without local server setup.